

Optimizing Trim Operations by In-Storage Processing

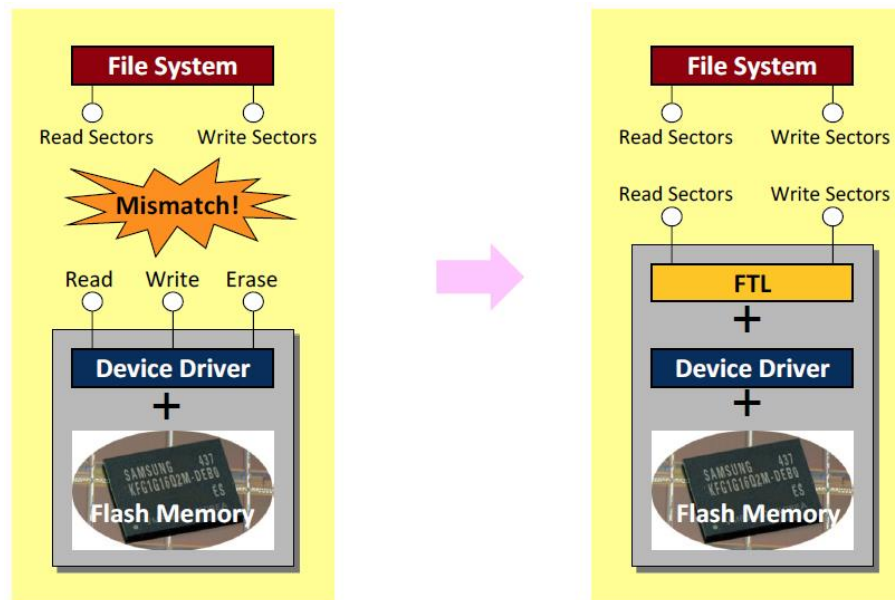
Taekoan Yoo, K. H. Han, D. K. Shin
Embedded Software Lab.

1. Introduction

2

15

- **Characteristics of SSDs (Solid-State Drives)**
 - Erase-before-write
 - Read/program and erase operations use different granularities.
 - Read/Program: 4KB or 8KB : page
 - erase 64pages or 128pages : block
 - No erase interface is exposed to the host
→ FTL (Flash Translation Layer) is required.



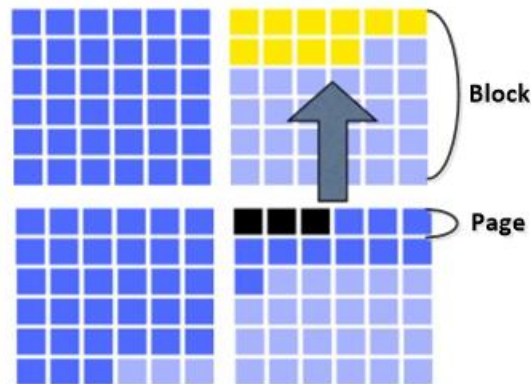
1. Introduction

3

15

- **FTL Garbage Collection (GC):** Moves valid data to another block and then erases the original block.

Garbage Collection



- **Trim for GC:** When a file is deleted, the host needs to inform the device that the corresponding data is no longer valid.
- Conventional Trim operates at the extent granularity, resulting in high I/O traffic.
 - ➔ **vectored-Trim(vTrim) & file-based-Trim(fTrim)**

2. Background

4

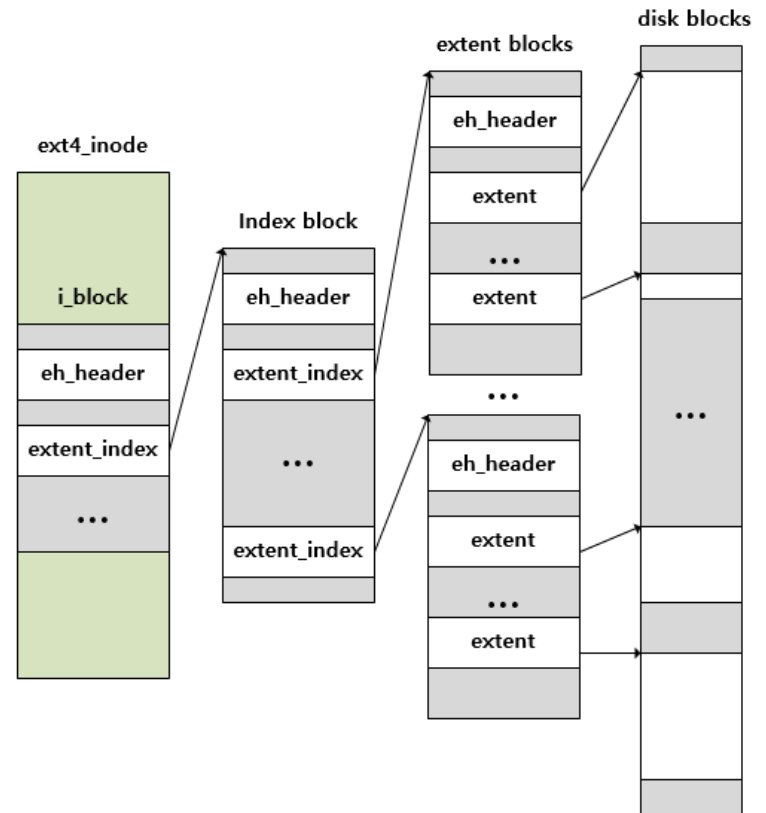
15

- **EXT4 filesystem**

- Manages data in 4 KB blocks.
 - **Index node:** <key, address of extent block>
 - **Extent (leaf) node:** <starting logical block address, number of contiguous logical blocks>
- File deletion: Sends information for all extents to the device through Trim commands.

- **SSD FTL**

- Marks the corresponding entries in the L2P mapping table as invalid.



2. Background

5

15

- **Original Trim**

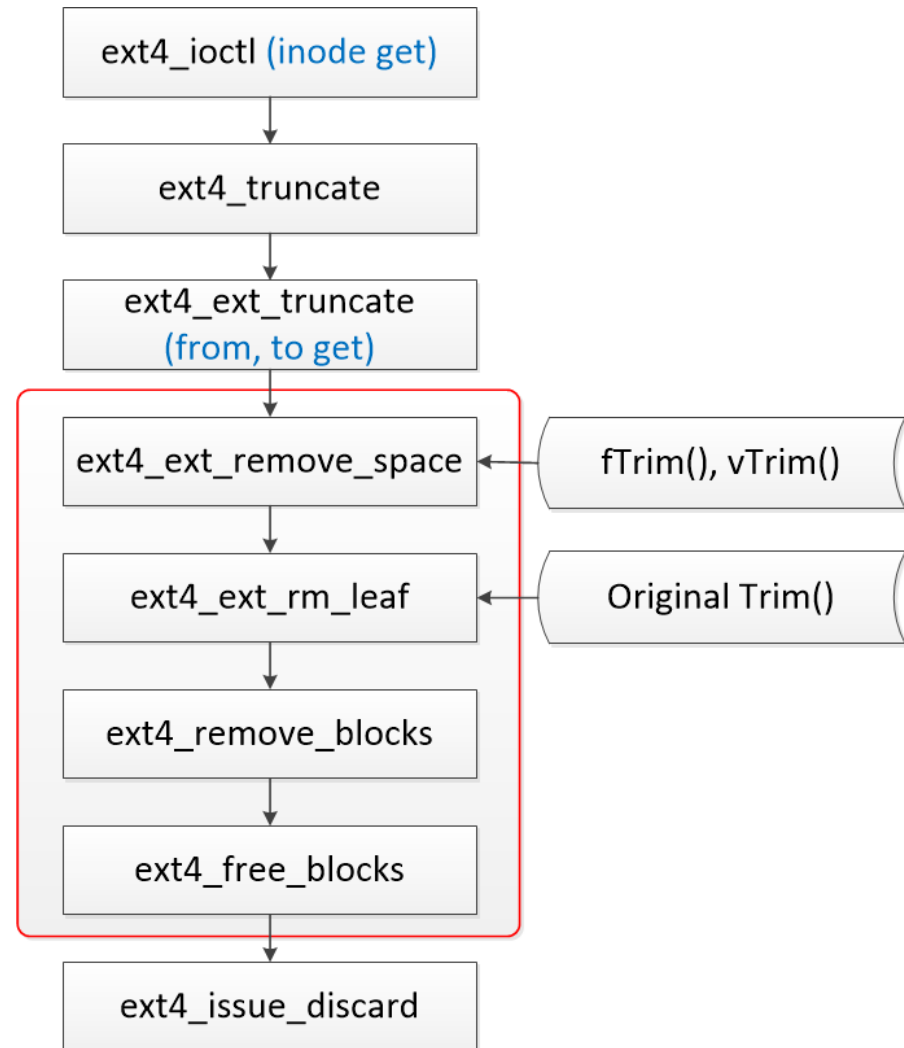
- Sends a Trim command to the device for each extent.

- **vTrim**

- Batches multiple Trim requests and sends them to the device in a single I/O.

- **fTrim**

- Sends only the file's inode information to the device.

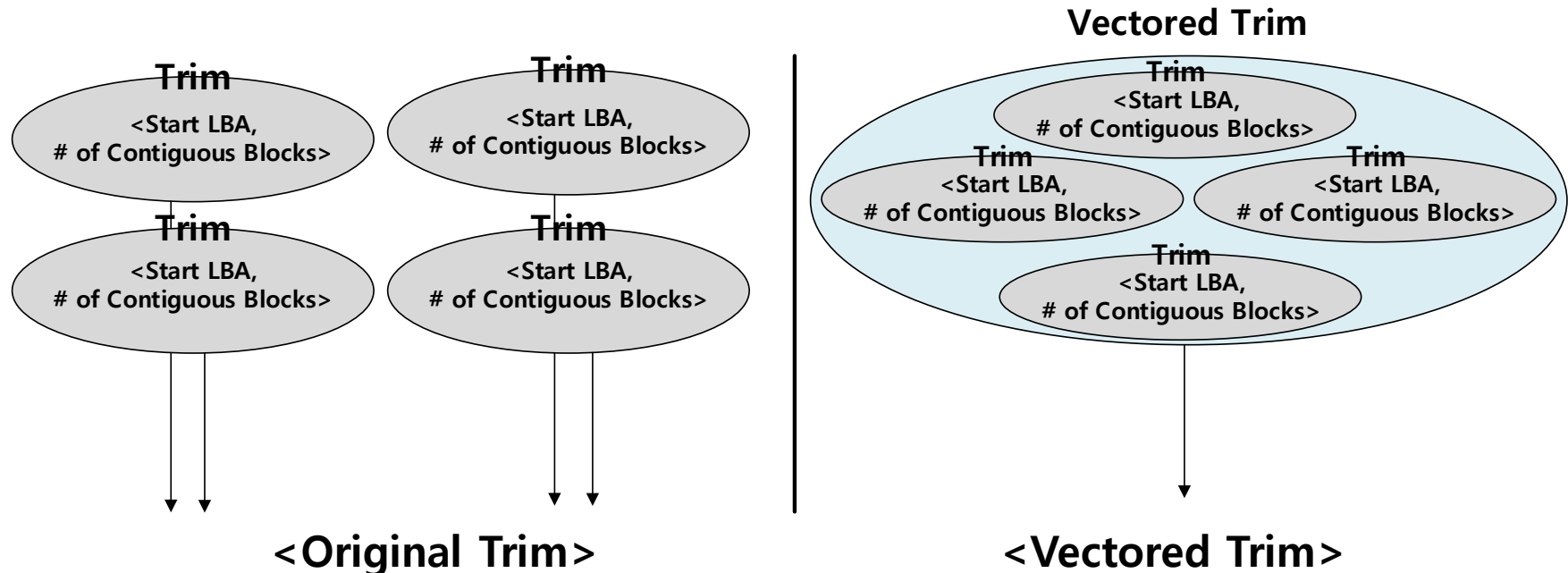


3. Vectored Trim

6

15

- Sends a structure containing multiple Trim requests to the device at once.
- Collects information from all extents in the extent tree, records the total number of extents, and sends them to the device in a single request.

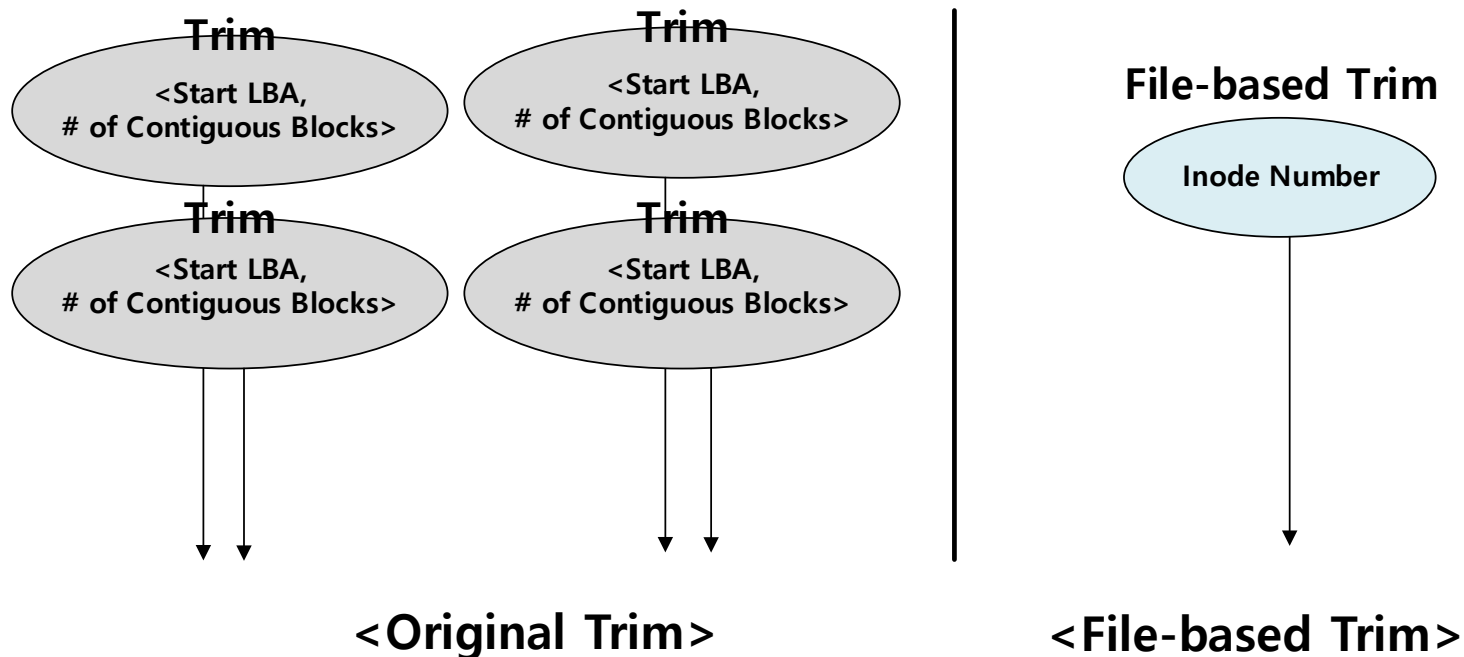


4. File-based Trim

7

15

- Sends only the inode number of the deleted file to the device.
- The SSD directly examines the file system to locate and process the corresponding extents.

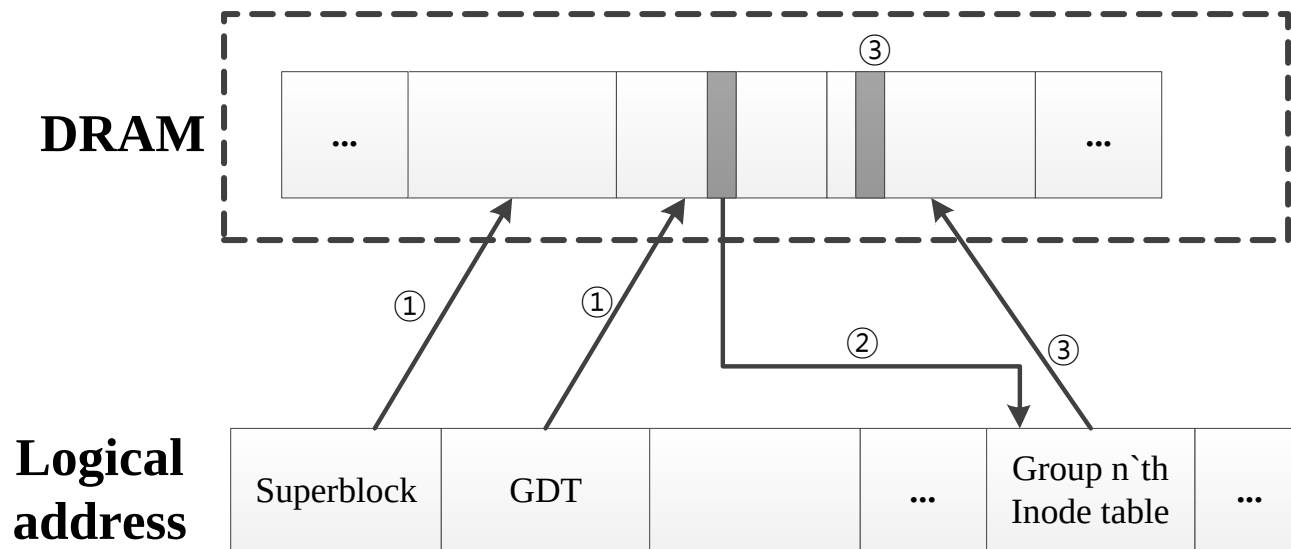


4. File-based Trim

8

15

- ① Read the superblock and group descriptor into DRAM.
- ② Use them to locate the inode table and the target inode.
- ③ Load the target inode into DRAM and traverse its extent tree.

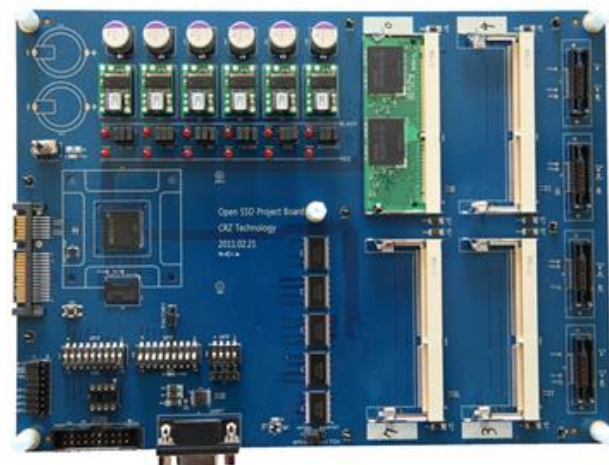


5. Experiments

9

15

- **Experimental Setup**
- **Test PC setting**
 - Intel i5-3580 Quad core CPU
 - 8GB DRAM
 - OS: Ubuntu 14.04 with Linux kernel 3.13.0
- **Device** : 32 GB OpenSSD, 16 GB EXT4
- **Experimental Method**
 - Measure the time required to delete a 10 MB file consisting of 160 extents.



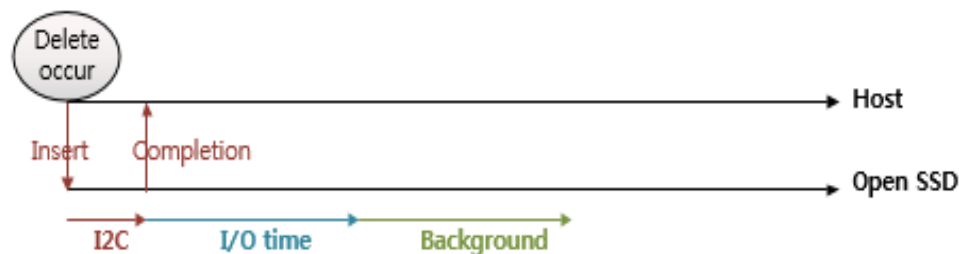
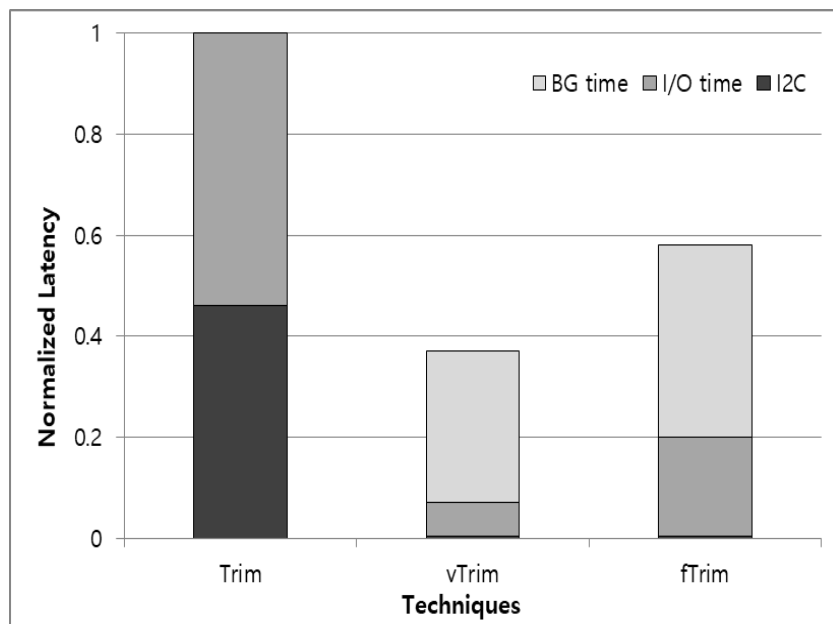
5. Experiments

10

15

- **File Deletion Latency Breakdown**

- Stages
 - I2C: Insert to Completion
 - I/O time: Device processing time
 - BG time: Background processing time during idle periods
- vTrim and fTrim significantly reduce I2C latency.
- fTrim has a longer I/O time than vTrim because EXT4 metadata must be loaded into DRAM.



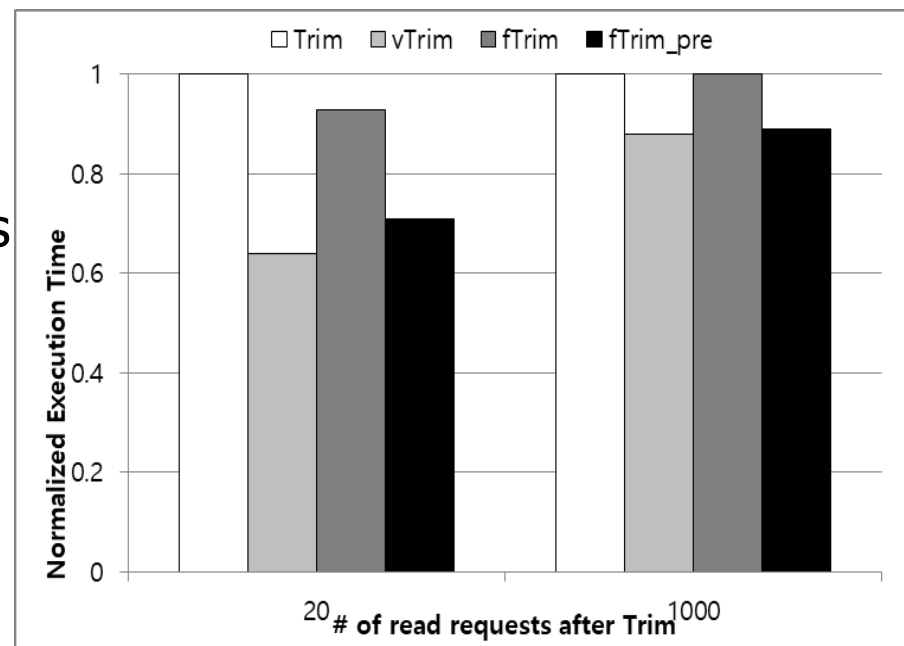
5. Experiments

11

15

• Execution Time Comparison with Multiple Read Requests

- Both vTrim and fTrim reduce execution time.
- As the number of read requests increases, Trim accounts for a smaller fraction of the total execution time, making the performance improvement less noticeable.
- Preemption further reduces execution time.



	(%)	Read 20	Read 1000
	Original	1	1
	<u>vTrim</u>	0.64	0.88
Em	<u>fTrim</u>	0.71	0.89

6. Conclusion

12

15

- **1. vTrim:** By batching multiple extents into a single request, vTrim reduces interface overhead. Most processing is handled by the relatively faster host, leaving only lightweight operations to the SSD.
- **2. fTrim:** By sending only the file's inode number, fTrim reduces interface overhead.
- **3. <Future work> :** Improve Trim performance under heavy user I/O when multiple files are deleted simultaneously.